

Part 7: System Design & Software Architecture

Chapter 1: Designing Systems That Serve Millions

The company had mastered cloud infrastructure.

They had:

- * Scalable databases
- * Container platforms
- * Automated deployments
- * Monitoring systems
- * Security controls

But a new challenge arrived.

The CEO walked into the engineering meeting.

> "We are launching a global platform."

Everyone looked interested.

The CEO continued:

> "We expect millions of users in the first year."

The room became quiet.

Emma smiled.

> "Building something that works for one thousand users is easy."

Chris nodded.

> "Building something that works for millions is a different problem."

What Is System Design?

Sarah opened a blank architecture diagram.

She wrote:

****System Design****

She explained:

> "System design is the process of defining how different components work together to solve a large problem."

Emma asked,

> "So it is bigger than writing code?"

Sarah nodded.

> "Much bigger."

It includes:

- * Requirements
- * Architecture
- * Data flow
- * Scalability
- * Reliability
- * Security
- * Performance

Functional Requirements

Maya started the planning session.

> "First, what should the system do?"

The product team answered:

Users should be able to:

- * Create accounts
- * Upload content
- * Search information
- * Communicate
- * Receive notifications

Sarah smiled.

> "These are functional requirements."

Non-Functional Requirements

Chris added another category.

> "Now we ask how the system should behave."

Examples:

- * Fast response time
- * High availability
- * Security
- * Scalability
- * Reliability

Emma nodded.

> "These are non-functional requirements."

Estimation

Before designing, the team estimated usage.

Maya asked:

> "How many users?"

Product team:

100 million users.

Chris asked:

> "How many requests per second?"

The team calculated:

- * Daily active users
- * Requests per user
- * Storage growth
- * Bandwidth requirements

Emma smiled.

> "We design based on numbers, not guesses."

Traffic Estimation

Sarah wrote:

****Requests Per Second (RPS)****

She explained:

> "We need to know how much traffic the system must handle."

Example:

1 million users

x

100 requests per day

=

100 million requests daily

Storage Estimation

The team calculated data growth.

Images.

Videos.

Messages.

Logs.

Emma asked:

> "Why estimate storage?"

Chris replied:

> "Because architecture decisions depend on scale."

Latency

The team discussed speed.

Sarah wrote:

****Latency****

She explained:

> "Latency is the time required for a request to receive a response."

Emma nodded.

> "Users notice delays."

Throughput

Chris added:

****Throughput****

> "Throughput measures how much work the system can process in a given time."

Examples:

- * Requests per second
- * Messages per second
- * Transactions per second

Availability

Maya asked:

> "What happens if one server fails?"

Sarah answered:

> "The system should continue working."

She wrote:

****High Availability****

Reliability

Chris explained:

> "Availability means the system is running."

> "Reliability means it performs correctly over time."

Emma nodded.

> "A system can be available but unreliable."

Scalability

The company expected rapid growth.

Sarah explained:

> "Scalability is the ability to handle increasing demand."

Two approaches:

Vertical Scaling

Bigger machines.

Horizontal Scaling

More machines.

Load Balancing

The team created the first architecture.

Users

↓

Load Balancer

↓

Application Servers

Emma asked:

> "Why do we need a load balancer?"

Chris answered:

> "To distribute traffic across multiple servers."

Stateless Services

Sarah reviewed the application servers.

She removed stored session data.

Emma asked:

> "Where did the sessions go?"

Chris replied:

> "Into a shared storage system."

Sarah explained:

> "Stateless services scale more easily."

Stateful Services

The database remained different.

Chris explained:

> "Some systems need memory."

Examples:

- * Databases
- * Message queues
- * Session stores

Emma nodded.

> "Those are stateful services."

Caching

The team noticed repeated requests.

Millions of users requested the same information.

Sarah introduced:

****Cache****

Cache Strategy

Chris explained:

Instead of:

User

↓

Database

The system became:

User

↓

Cache

↓

Database

Emma smiled.

> "Frequently used data becomes faster to access."

Cache Invalidation

James asked:

> "What happens when data changes?"

Sarah laughed.

> "The hardest problem in computer science."

She explained:

> "Cache invalidation decides when cached data should be updated or removed."

Content Delivery Network (CDN)

The company stored images globally.

Chris added:

****CDN****

Emma asked:

> "Why?"

Sarah explained:

> "A CDN delivers content from servers closer to users."

Database Choice

The team discussed storage.

Different needs:

User profiles.

Messages.

Analytics.

Relationships.

Chris said:

> "The database choice depends on the problem."

SQL vs NoSQL

Sarah compared approaches.

SQL:

- * Structured data
- * Relationships
- * Transactions

NoSQL:

- * Flexible schemas
- * Massive scale
- * High throughput

Emma nodded.

> "There is no universal best database."

API Design

The team designed communication between clients and services.

Sarah introduced:

****API Contract****

Emma asked:

> "What does it define?"

Chris replied:

- * Endpoints
- * Requests
- * Responses
- * Authentication rules

REST API

The first design used REST.

Example:

GET /users/123

POST /orders

Emma smiled.

> "Simple and widely used."

GraphQL

Another team preferred:

GraphQL.

Sarah explained:

> "Clients request exactly the data they need."

The First Architecture

After several days, the design looked like this:

Users

↓

CDN

↓

Load Balancer

↓

API Gateway

↓

Stateless Services

↓

Cache

↓

Databases

↓

Message Queue

↓

Background Workers

The team reviewed the design.

Wrapping Up

Maya summarized the first system design lesson.

- ✓ System Design
- ✓ Functional Requirements
- ✓ Non-Functional Requirements
- ✓ Estimation
- ✓ Traffic Estimation
- ✓ Storage Estimation
- ✓ Latency
- ✓ Throughput
- ✓ Availability
- ✓ Reliability
- ✓ Scalability
- ✓ Vertical Scaling
- ✓ Horizontal Scaling
- ✓ Load Balancer
- ✓ Stateless Service
- ✓ Stateful Service
- ✓ Cache
- ✓ Cache Invalidation
- ✓ CDN
- ✓ SQL
- ✓ NoSQL
- ✓ API Contract

✓ REST API

✓ GraphQL

Maya closed the meeting.

> "A great system is not created by adding more technology."

Emma nodded.

> "It starts with understanding the problem."

Chris smiled.

> "Architecture is about making the right trade-offs."

The team looked at the empty architecture board.

Millions of users.

Billions of requests.

A huge challenge.

But they had learned the most important lesson:

****Before building a system, learn how to think like a system designer.****

Part 7: System Design & Software Architecture

Chapter 2: Designing a Global Social Platform

The company's new platform had a clear mission:

Connect people around the world.

Users would be able to:

- * Create profiles
- * Follow other users
- * Share posts
- * Like content
- * Comment
- * Receive notifications
- * Communicate in real time

The product team expected explosive growth.

Maya looked at the engineering team.

> "We are not building a simple website."

She pointed at the architecture board.

> "We are building a global social platform."

Emma smiled.

> "The challenge is not making it work."

Chris nodded.

> "The challenge is making it work for hundreds of millions of users."

Understanding the Requirements

Before writing code, Sarah asked the first question:

> "What are the core features?"

The product team listed:

- * User registration
- * User profiles
- * Following relationships
- * Creating posts
- * Viewing feeds
- * Likes and comments
- * Notifications
- * Search

Sarah nodded.

> "These are the functional requirements."

Non-Functional Requirements

Chris added:

> "Now let's define system goals."

The platform needed:

- * Low latency
- * High availability
- * Global scalability
- * Fault tolerance
- * Data security

Emma smiled.

> "The system must stay fast even when millions of people use it."

User Service

The first component was the user system.

It handled:

- * Registration
- * Login
- * Profiles
- * Settings

Sarah designed:

Users

↓

User Service

↓

User Database

User IDs

James asked:

> "How do we identify every user?"

Chris explained:

> "Each user receives a unique identifier."

The system used:

- * Numeric IDs
- * UUIDs
- * Distributed ID generators

Emma nodded.

> "IDs must remain unique even across many servers."

Social Graph

The team reached the most important feature.

Following relationships.

Emma asked:

> "How do we store who follows whom?"

Sarah explained:

> "This is called the social graph."

Example:

Alice follows Bob.

Bob follows Charlie.

Charlie follows David.

Graph Database vs Traditional Database

James asked:

> "Should we use a graph database?"

Chris explained:

"Graph databases are excellent for relationships."

But at massive scale:

- * Storage requirements
- * Query patterns
- * Performance

must be carefully considered.

Emma nodded.

> "The best database depends on the access pattern."

Creating Posts

The team designed the posting flow.

User creates post.

↓

Post Service

↓

Storage System

↓

Feed System

Post Storage

Chris explained:

Posts contained:

- * Post ID
- * User ID
- * Text
- * Media references
- * Timestamp

Sarah added:

> "Large media files should not live directly inside the database."

Object Storage

Images and videos were stored separately.

Flow:

User Upload

↓

Media Service

↓

Object Storage

↓

CDN

Emma smiled.

> "The database stores metadata, not huge files."

The Feed Problem

The biggest challenge arrived.

The home feed.

A user follows 5,000 people.

When they open the app:

Which posts should they see?

How fast?

How should content be ranked?

Timeline Generation

Chris explained two approaches.

Pull Model

When the user opens the app:

System collects posts from followed users.

Emma asked:

> "Generate the feed at request time?"

Chris nodded.

Push Model

When someone creates a post:

The system pushes it to followers.

Sarah explained:

> "The feed is prepared before users request it."

Fan-Out

The team discussed the push model.

One celebrity had 100 million followers.

Emma asked:

> "Do we create 100 million copies instantly?"

Chris smiled.

> "That's the fan-out problem."

Fan-Out on Write

When a user posts:

System sends the post to followers' feeds.

Advantages:

- * Fast reads

Disadvantages:

- * Expensive writes

Fan-Out on Read

When a user opens the app:

System collects posts dynamically.

Advantages:

- * Simple writes

Disadvantages:

- * Slower reads

Hybrid Approach

Sarah designed a compromise.

Normal users:

Fan-out on write.

Celebrity accounts:

Fan-out on read.

Emma smiled.

> "Different users require different strategies."

Feed Ranking

The newest post was not always the best.

The team created a ranking system.

Factors:

- * User relationship

- * Engagement

- * Time

- * Relevance

- * User interests

Recommendation System

The machine learning team joined.

They introduced:

- * User behavior analysis

- * Content similarity

- * Prediction models

Emma asked:

> "The system learns what users like?"

Sarah nodded.

> "Exactly."

Caching the Feed

The team noticed users opened feeds frequently.

Chris added:

****Feed Cache****

User

↓

Feed Cache

↓

Post Storage

Emma smiled.

> "Fast access for common requests."

Real-Time Notifications

Users needed instant updates.

Examples:

* Someone liked your post

* Someone followed you

* Someone commented

Sarah designed:

Event

↓

Notification Service

↓

User Device

Message Queue

Chris inserted a queue.

Post Created

↓

Message Queue

↓

Notification Workers

Emma asked:

> "Why add a queue?"

Chris replied:

> "To separate services and handle traffic spikes."

Background Workers

Some tasks did not need immediate processing.

Examples:

- * Sending notifications
- * Processing images
- * Updating recommendations

Sarah created background workers.

Real-Time Communication

The company wanted instant messaging.

The old HTTP request model was not enough.

James asked:

> "How do we keep a live connection?"

Chris answered:

> "WebSockets."

WebSockets

Unlike normal requests:

Client

↔

Server

The connection stays open.

Emma smiled.

> "Both sides can communicate instantly."

Scaling WebSocket Servers

Millions of users created millions of connections.

Sarah added:

- * Connection management
- * Session storage
- * Load balancing

Database Sharding

The user database became enormous.

Chris divided users across databases.

Shard 1:

Users A-F

Shard 2:

Users G-M

Shard 3:

Users N-Z

Emma nodded.

> "Horizontal database scaling."

Consistency Trade-Off

The team faced a decision.

Should every action appear instantly everywhere?

Or should some updates arrive slightly later?

Sarah explained:

> "Distributed systems require trade-offs."

Examples:

Strong consistency:

More accurate.

Eventual consistency:

More scalable.

The Final Architecture

After months of design:

Users

↓

CDN

↓

API Gateway

↓

Microservices

↓

Feed Service

↓

Cache

↓

Databases

↓

Message Queue

↓

Background Workers

↓

Notification System

↓

Machine Learning Ranking

The platform was ready.

Wrapping Up

Maya reviewed the social platform design.

- ✓ User Service
- ✓ User ID
- ✓ Social Graph
- ✓ Post Service
- ✓ Object Storage
- ✓ CDN
- ✓ Timeline
- ✓ Feed Generation
- ✓ Pull Model
- ✓ Push Model
- ✓ Fan-Out on Write
- ✓ Fan-Out on Read
- ✓ Hybrid Feed Strategy
- ✓ Ranking System
- ✓ Recommendation System
- ✓ Feed Cache
- ✓ Notification Service
- ✓ Message Queue
- ✓ Background Workers
- ✓ WebSockets
- ✓ Database Sharding
- ✓ Consistency Trade-Off

The team looked at the completed architecture.

Emma smiled.

> "A social network looks simple from the outside."

Chris nodded.

> "But behind every button is a massive distributed system."

Sarah closed the design document.

> "The hardest part of system design is not creating components."

She smiled.

> "It's deciding how those components cooperate at scale."

The team was ready for the next challenge.

A system used by millions.

A system where every millisecond mattered.

A system where architecture decisions could determine success or failure.

Part 7: System Design & Software Architecture

Chapter 3: Designing a Global Payment System

The social platform was a success.

Millions of users were active every day.

But the company wanted to build something even more challenging.

A financial platform.

The CEO entered the engineering meeting.

> "We want users to send money, receive payments, and purchase products globally."

The room became quiet.

Emma looked at Chris.

> "Payments are different."

Chris nodded.

> "A social media bug is annoying."

> "A payment bug can lose money."

Understanding the Problem

Before designing anything, Sarah asked:

> "What must the system guarantee?"

The product team answered:

Users should be able to:

- * Create payment accounts
- * Add payment methods
- * Send money
- * Receive money
- * Buy products
- * View transaction history

Sarah nodded.

> "Those are functional requirements."

Non-Functional Requirements

Chris added:

The system must provide:

- * Extremely high reliability
- * Strong security
- * Accurate balances
- * Low latency
- * Auditability
- * Fraud protection

Emma smiled.

> "Money requires trust."

Payment Architecture Overview

The first design:

User

↓

Payment API

↓

Payment Service

↓

Transaction System

↓

Ledger Database

↓

External Payment Providers

Payment Service

The payment service handled:

- * Payment requests
- * Validation
- * Authorization
- * Processing
- * Status updates

James asked:

> "Is this where balances are stored?"

Sarah shook her head.

> "No."

The Ledger System

Sarah wrote:

****Ledger****

She explained:

> "The ledger is the source of truth for financial records."

Example:

Account A:

-100 dollars

Account B:

+100 dollars

Emma nodded.

> "Every movement of money must be recorded."

Double-Entry Bookkeeping

Chris introduced a financial concept.

Every transaction has two sides.

Example:

Customer pays \$50.

Debit:

Customer Account

Credit:

Merchant Account

Emma smiled.

> "Money cannot disappear or appear randomly."

Transaction IDs

Every payment received a unique identifier.

Example:

TXN-928374

James asked:

> "Why?"

Chris replied:

> "To track, search, and reconcile every transaction."

Idempotency

The team discussed a serious problem.

A customer clicks:

"Pay"

The network fails.

The user clicks again.

Emma asked:

> "Could they be charged twice?"

Sarah smiled.

> "That's why we need idempotency."

Idempotency Keys

Each payment request includes a unique key.

Example:

payment_request_12345

If the same request arrives twice:

First request:

Process payment.

Second request:

Return previous result.

Emma nodded.

> "Same operation, same outcome."

Payment States

The team created payment states.

Created

↓

Processing

↓

Completed

or

Failed

or

Cancelled

Chris explained:

> "State transitions must be carefully controlled."

Distributed Transactions

Payments involved multiple systems.

Example:

Payment System

↓

Bank Provider

↓

Merchant System

Emma asked:

> "How do we keep everything consistent?"

Sarah smiled.

> "That is one of the hardest problems in distributed systems."

Two-Phase Commit

James suggested:

> "Can we use two-phase commit?"

Chris explained:

Phase 1:

Prepare.

Phase 2:

Commit.

But he added:

> "At massive scale, it can create performance problems."

Saga Pattern

Sarah introduced another approach.

****Saga Pattern****

A transaction becomes a sequence of smaller steps.

Example:

1. Reserve money.
2. Process payment.
3. Confirm purchase.
4. Update order.

If something fails:

Compensating actions undo previous steps.

Emma nodded.

> "A distributed rollback."

Payment Gateway

The company connected to external providers.

Sarah added:

****Payment Gateway****

It handled communication with:

- * Banks
- * Card networks
- * Digital wallets

External APIs

Chris explained:

External systems could fail.

Therefore:

- * Timeouts
- * Retries
- * Circuit breakers

were required.

Fraud Detection

Security engineers joined the meeting.

They introduced:

Fraud Detection System

It analyzed:

- * User behavior
- * Location
- * Transaction amount
- * Device information
- * Previous activity

Machine Learning in Payments

The AI team built models.

During training:

The model learns fraud patterns.

During inference:

The model predicts risk.

Emma smiled.

> "The system makes decisions in real time."

Risk Score

Each payment received a score.

Example:

Low Risk:

Approve automatically.

High Risk:

Require verification.

Authentication

For suspicious payments:

The system requested:

- * Password

- * MFA

- * Additional verification

Audit Logs

Maya asked:

> "How do we investigate problems?"

Sarah answered:

> "Audit logs."

They recorded:

- * Who performed an action

- * When it happened

- * What changed

Reconciliation

At the end of each day:

Internal records

↓

Compared with

Bank records

Chris explained:

> "This process is called reconciliation."

Emma nodded.

> "Making sure every number matches."

Event-Driven Payments

The team improved the design.

Payment Completed

↓

Event

↓

Notification Service

↓

Receipt Service

↓

Analytics System

Sarah smiled.

> "Events keep services loosely coupled."

Database Design

The payment database required:

- * Strong consistency
- * Transactions
- * ACID guarantees
- * Careful indexing

James asked:

> "Can we use a normal database?"

Chris replied:

> "Possibly, but financial workloads require careful design."

Disaster Recovery

The team created:

- * Backups
- * Replication
- * Failover
- * Recovery plans

Emma nodded.

> "Financial systems cannot simply disappear."

Security Architecture

The payment platform included:

- * Encryption
- * Tokenization
- * Access control
- * Secrets management
- * Monitoring

Final Payment Architecture

The completed system:

Users

↓

Payment API

↓

Payment Service

↓

Fraud Detection

↓

Transaction Engine

↓

Ledger System

↓

Database

↓

External Payment Networks

↓

Audit + Reporting

Wrapping Up

Maya summarized the payment system concepts.

- ✓ Payment Service
- ✓ Ledger
- ✓ Double-Entry Bookkeeping
- ✓ Transaction ID
- ✓ Idempotency
- ✓ Idempotency Key
- ✓ Payment States
- ✓ Distributed Transactions
- ✓ Two-Phase Commit
- ✓ Saga Pattern
- ✓ Payment Gateway
- ✓ External APIs
- ✓ Fraud Detection
- ✓ ML Risk Scoring
- ✓ Authentication
- ✓ Audit Logs
- ✓ Reconciliation

✅ Event-Driven Payments

✅ Financial Database Design

✅ Disaster Recovery

The team looked at the final architecture.

Emma smiled.

> "Payments are not just about moving money."

Chris nodded.

> "They are about proving that every movement of money is correct."

Sarah closed the design document.

> "In system design, correctness is often more important than speed."

The team had learned a powerful lesson:

When designing financial systems, trust is the most important feature.

Part 7: System Design & Software Architecture

Chapter 4: Designing a Global Streaming Platform

The payment system was live.

Millions of transactions were processed safely.

But the company wanted to enter another massive industry.

Entertainment.

The CEO announced the next project:

> "We are building a global video streaming platform."

The engineering team immediately understood the challenge.

Emma smiled.

> "Payments require correctness."

Chris nodded.

> "Streaming requires speed, storage, and global delivery."

Understanding the Requirements

Sarah started the design meeting.

> "What should users be able to do?"

The product team answered:

Users should be able to:

- * Create accounts
- * Browse videos
- * Search content
- * Watch videos
- * Upload videos
- * Rate content
- * Receive recommendations

Sarah nodded.

> "These are functional requirements."

Non-Functional Requirements

Chris added:

The platform must provide:

- * Fast video startup
- * High availability
- * Global scalability
- * Smooth playback
- * Low buffering
- * Data security

Emma smiled.

> "A user should press play and forget about the technology behind it."

High-Level Architecture

The first design:

Users

↓

Mobile/Web Applications

↓

API Gateway

↓

Video Platform Services

↓

Storage + Processing Systems

↓

Content Delivery Network

Video Upload Service

Creators uploaded videos.

The upload flow:

User Upload

↓

Upload Service

↓

Storage System

↓

Processing Pipeline

Emma asked:

> "Do we immediately make the video available?"

Sarah shook her head.

> "Not yet."

Video Processing Pipeline

A raw video file could be huge.

The system needed to process it.

Steps:

1. Validate file.
2. Extract metadata.
3. Encode video.
4. Generate thumbnails.
5. Store processed versions.

Video Encoding

Chris explained:

> "Different devices require different video formats."

Examples:

- * Mobile devices
- * TVs
- * Browsers
- * Tablets

The system created multiple versions.

Compression

Sarah explained:

> "Video files are extremely large."

Compression reduces:

- * File size
- * Storage cost
- * Network usage

while maintaining quality.

Transcoding

The team introduced:

****Transcoding****

Emma asked:

> "What happens during transcoding?"

Chris replied:

> "A video is converted from one format or quality level into another."

Example:

Original:

4K video

↓

Processed:

1080p

720p

480p

Adaptive Bitrate Streaming

The biggest challenge was different internet speeds.

A user on fast internet:

High quality.

A user on slow internet:

Lower quality.

Sarah introduced:

****Adaptive Bitrate Streaming****

How Adaptive Streaming Works

The video is divided into small segments.

The player chooses quality dynamically.

Example:

Network improves:

480p → 720p → 1080p

Network slows:

1080p → 720p → 480p

Emma smiled.

> "The video adapts while playing."

Video Storage

The company stored billions of video files.

Chris asked:

> "Should we store videos in the database?"

Everyone laughed.

Sarah explained:

> "Large media belongs in object storage."

Object Storage

The system stored:

- * Video files
- * Thumbnails
- * Audio tracks
- * Subtitles

The database stored:

- * Metadata
- * Ownership
- * Permissions

Content Delivery Network (CDN)

The team faced a global challenge.

A user in Asia should not download video from a server in America.

Chris introduced:

****CDN****

CDN Architecture

Original Content

↓

CDN Edge Servers

↓

Users

Emma smiled.

> "Content moves closer to users."

Edge Locations

Sarah explained:

> "CDN servers are distributed around the world."

Benefits:

- * Lower latency
- * Faster startup
- * Reduced origin load

Caching Strategy

Popular videos were requested millions of times.

The CDN cached them.

Chris explained:

> "Caching reduces pressure on the original storage system."

Metadata Service

The platform needed information about videos.

Examples:

- * Title
- * Description
- * Category
- * Creator
- * Views

Sarah created:

Video Metadata Service

Search System

Users needed to find videos.

The search architecture included:

- * Indexing
- * Ranking
- * Filtering

Emma asked:

> "Do we search directly in the database?"

Chris smiled.

> "Not at this scale."

Search Index

The platform created a dedicated search index.

Flow:

Video Created

↓

Indexing Service

↓

Search Engine

↓

User Query

Recommendation System

The company wanted personalized suggestions.

The AI team joined.

They analyzed:

- * Watch history
- * Likes
- * Search behavior
- * Similar users

Machine Learning Pipeline

The recommendation system had two stages.

Training

The model learns patterns.

Inference

The model generates recommendations.

Emma nodded.

> "The model predicts what users may enjoy."

Real-Time Recommendations

Some signals changed instantly.

Example:

A user watches many cooking videos.

The system updates recommendations.

Sarah explained:

> "Real-time features improve personalization."

View Tracking

The platform tracked:

- * Views
- * Watch duration
- * Completion rate
- * Engagement

Chris explained:

> "These metrics improve recommendations and business decisions."

Background Processing

Many tasks happened asynchronously.

Examples:

- * Video encoding
- * Thumbnail generation
- * Analytics processing

Sarah added:

****Message Queue****

Video Uploaded

↓

Queue

↓

Processing Workers

Scaling Video Processing

Millions of uploads required massive computing power.

The team used:

- * Distributed workers
- * Autoscaling
- * Parallel processing

Emma smiled.

> "Many machines process videos together."

Reliability Challenges

The team prepared for failures.

Possible problems:

- * Storage failure
- * Processing failure
- * Network problems

Solutions:

- * Replication
- * Retry mechanisms
- * Monitoring
- * Disaster recovery

Security

The platform protected:

- * User accounts
- * Private videos
- * Copyright content

Security included:

- * Authentication
- * Authorization
- * Encryption
- * Access control

Final Streaming Architecture

The completed system:

Users

↓

CDN

↓

API Gateway

↓

Video Services

↓

Metadata Database

↓

Object Storage

↓

Processing Pipeline

↓

Encoding Workers

↓

Recommendation System

↓

Analytics Platform

Wrapping Up

Maya summarized the streaming platform design.

- ✓ Video Upload Service
- ✓ Video Processing Pipeline
- ✓ Video Encoding
- ✓ Compression
- ✓ Transcoding
- ✓ Adaptive Bitrate Streaming
- ✓ Object Storage
- ✓ CDN
- ✓ Edge Locations
- ✓ Caching Strategy
- ✓ Metadata Service
- ✓ Search Index
- ✓ Recommendation System
- ✓ ML Training
- ✓ ML Inference
- ✓ View Tracking
- ✓ Message Queue

✅ Distributed Processing

✅ Autoscaling

✅ Disaster Recovery

The team reviewed the final architecture.

Emma smiled.

> "A video player looks simple."

Chris nodded.

> "But behind one play button is a global distributed system."

Sarah closed the design document.

> "System design is about understanding the hidden complexity behind simple experiences."

The team was ready for the next challenge.

A system where every second mattered.

A system that connected people instantly.

A real-time communication platform.

Part 7: System Design & Software Architecture

Chapter 4: Designing a Global Streaming Platform

The payment system was live.

Millions of transactions were processed safely.

But the company wanted to enter another massive industry.

Entertainment.

The CEO announced the next project:

> "We are building a global video streaming platform."

The engineering team immediately understood the challenge.

Emma smiled.

> "Payments require correctness."

Chris nodded.

> "Streaming requires speed, storage, and global delivery."

Understanding the Requirements

Sarah started the design meeting.

> "What should users be able to do?"

The product team answered:

Users should be able to:

- * Create accounts
- * Browse videos
- * Search content
- * Watch videos
- * Upload videos
- * Rate content
- * Receive recommendations

Sarah nodded.

> "These are functional requirements."

Non-Functional Requirements

Chris added:

The platform must provide:

- * Fast video startup
- * High availability
- * Global scalability
- * Smooth playback
- * Low buffering
- * Data security

Emma smiled.

> "A user should press play and forget about the technology behind it."

High-Level Architecture

The first design:

Users

↓

Mobile/Web Applications

↓

API Gateway

↓

Video Platform Services

↓

Storage + Processing Systems

↓

Content Delivery Network

Video Upload Service

Creators uploaded videos.

The upload flow:

User Upload

↓

Upload Service

↓

Storage System

↓

Processing Pipeline

Emma asked:

> "Do we immediately make the video available?"

Sarah shook her head.

> "Not yet."

Video Processing Pipeline

A raw video file could be huge.

The system needed to process it.

Steps:

1. Validate file.
2. Extract metadata.
3. Encode video.

4. Generate thumbnails.
5. Store processed versions.

Video Encoding

Chris explained:

> "Different devices require different video formats."

Examples:

- * Mobile devices
- * TVs
- * Browsers
- * Tablets

The system created multiple versions.

Compression

Sarah explained:

> "Video files are extremely large."

Compression reduces:

- * File size
- * Storage cost
- * Network usage

while maintaining quality.

Transcoding

The team introduced:

****Transcoding****

Emma asked:

> "What happens during transcoding?"

Chris replied:

> "A video is converted from one format or quality level into another."

Example:

Original:

4K video

↓

Processed:

1080p

720p

480p

Adaptive Bitrate Streaming

The biggest challenge was different internet speeds.

A user on fast internet:

High quality.

A user on slow internet:

Lower quality.

Sarah introduced:

****Adaptive Bitrate Streaming****

How Adaptive Streaming Works

The video is divided into small segments.

The player chooses quality dynamically.

Example:

Network improves:

480p → 720p → 1080p

Network slows:

1080p → 720p → 480p

Emma smiled.

> "The video adapts while playing."

Video Storage

The company stored billions of video files.

Chris asked:

> "Should we store videos in the database?"

Everyone laughed.

Sarah explained:

> "Large media belongs in object storage."

Object Storage

The system stored:

- * Video files
- * Thumbnails
- * Audio tracks
- * Subtitles

The database stored:

- * Metadata
- * Ownership
- * Permissions

Content Delivery Network (CDN)

The team faced a global challenge.

A user in Asia should not download video from a server in America.

Chris introduced:

****CDN****

CDN Architecture

Original Content

↓

CDN Edge Servers

↓

Users

Emma smiled.

> "Content moves closer to users."

Edge Locations

Sarah explained:

> "CDN servers are distributed around the world."

Benefits:

- * Lower latency
- * Faster startup
- * Reduced origin load

Caching Strategy

Popular videos were requested millions of times.

The CDN cached them.

Chris explained:

> "Caching reduces pressure on the original storage system."

Metadata Service

The platform needed information about videos.

Examples:

- * Title
- * Description
- * Category
- * Creator
- * Views

Sarah created:

Video Metadata Service

Search System

Users needed to find videos.

The search architecture included:

- * Indexing
- * Ranking
- * Filtering

Emma asked:

> "Do we search directly in the database?"

Chris smiled.

> "Not at this scale."

Search Index

The platform created a dedicated search index.

Flow:

Video Created

↓

Indexing Service

↓

Search Engine

↓

User Query

Recommendation System

The company wanted personalized suggestions.

The AI team joined.

They analyzed:

- * Watch history
- * Likes
- * Search behavior
- * Similar users

Machine Learning Pipeline

The recommendation system had two stages.

Training

The model learns patterns.

Inference

The model generates recommendations.

Emma nodded.

> "The model predicts what users may enjoy."

Real-Time Recommendations

Some signals changed instantly.

Example:

A user watches many cooking videos.

The system updates recommendations.

Sarah explained:

```
> "Real-time features improve personalization."
```

```
# View Tracking
```

The platform tracked:

- * Views
- * Watch duration
- * Completion rate
- * Engagement

Chris explained:

```
> "These metrics improve recommendations and business decisions."
```

```
# Background Processing
```

Many tasks happened asynchronously.

Examples:

- * Video encoding
- * Thumbnail generation
- * Analytics processing

Sarah added:

```
**Message Queue**
```

Video Uploaded

↓

Queue

↓

Processing Workers

```
# Scaling Video Processing
```

Millions of uploads required massive computing power.

The team used:

- * Distributed workers
- * Autoscaling
- * Parallel processing

Emma smiled.

> "Many machines process videos together."

Reliability Challenges

The team prepared for failures.

Possible problems:

- * Storage failure
- * Processing failure
- * Network problems

Solutions:

- * Replication
- * Retry mechanisms
- * Monitoring
- * Disaster recovery

Security

The platform protected:

- * User accounts
- * Private videos
- * Copyright content

Security included:

- * Authentication
- * Authorization
- * Encryption
- * Access control

Final Streaming Architecture

The completed system:

Users

↓

CDN

↓

API Gateway

↓

Video Services

↓

Metadata Database

↓

Object Storage

↓

Processing Pipeline

↓

Encoding Workers

↓

Recommendation System

↓

Analytics Platform

Wrapping Up

Maya summarized the streaming platform design.

- ✅ Video Upload Service
- ✅ Video Processing Pipeline
- ✅ Video Encoding
- ✅ Compression
- ✅ Transcoding
- ✅ Adaptive Bitrate Streaming
- ✅ Object Storage
- ✅ CDN
- ✅ Edge Locations
- ✅ Caching Strategy
- ✅ Metadata Service
- ✅ Search Index
- ✅ Recommendation System
- ✅ ML Training

- ✓ ML Inference
- ✓ View Tracking
- ✓ Message Queue
- ✓ Distributed Processing
- ✓ Autoscaling
- ✓ Disaster Recovery

The team reviewed the final architecture.

Emma smiled.

> "A video player looks simple."

Chris nodded.

> "But behind one play button is a global distributed system."

Sarah closed the design document.

> "System design is about understanding the hidden complexity behind simple experiences."

The team was ready for the next challenge.

A system where every second mattered.

A system that connected people instantly.

A real-time communication platform.

Part 7: System Design & Software Architecture

Chapter 5: Designing a Real-Time Chat System

The streaming platform became one of the company's biggest products.

Millions of users watched videos every day.

But the company wanted to build something even more personal.

A global messaging platform.

The CEO entered the engineering meeting.

> "Users should be able to communicate instantly from anywhere in the world."

Emma looked at the architecture board.

> "Instant communication sounds simple."

Chris smiled.

> "Sending one message is simple."

He pointed at the empty diagram.

> "Delivering billions of messages reliably is the real challenge."

Understanding the Requirements

Sarah started the discussion.

> "First, what features do users need?"

The product team listed:

- * Send messages
- * Receive messages instantly
- * Create group chats
- * See online status
- * Receive notifications
- * Send images and files
- * View message history

Sarah nodded.

> "These are functional requirements."

Non-Functional Requirements

Chris added:

The system must provide:

- * Very low latency
- * High availability
- * Message reliability
- * Scalability
- * Data privacy
- * Global availability

Emma smiled.

> "A message should arrive quickly and safely."

High-Level Architecture

The team created the first design:

Users

↓

Mobile/Web Applications

↓

API Gateway

↓

Chat Services

↓

Message Infrastructure

↓

Storage Systems

The Connection Problem

James asked:

> "How do users receive messages instantly?"

Chris explained:

Traditional HTTP works like this:

Client:

"Do you have new messages?"

Server:

"No."

Client:

"Do you have new messages?"

Server:

"No."

Emma laughed.

> "That wastes resources."

Long Polling

Sarah introduced an early solution.

The client keeps a request open.

When a message arrives:

Server responds.

Emma nodded.

> "Better than constant checking."

But Chris added:

> "Still not ideal for millions of users."

WebSockets

The team moved to a better solution.

****WebSocket****

A persistent connection between:

Client

↔

Server

Emma smiled.

> "The server can push messages immediately."

Connection Servers

Millions of users created millions of connections.

Sarah designed:

Connection Service

Responsibilities:

- * Maintain active connections
- * Route messages
- * Track users

Presence System

Users wanted to know:

- * Online
- * Offline
- * Last seen

James asked:

> "How do we track that?"

Chris explained:

The system monitors:

- * Active connections
- * Heartbeats
- * User activity

Heartbeats

The client periodically sends signals.

Example:

"I'm still connected."

If heartbeats stop:

The user becomes offline.

Emma nodded.

> "A simple idea with huge scale challenges."

Sending a Message

The message flow:

User A

↓

Chat Server

↓

Message Queue

↓

Storage

↓

User B

Emma asked:

> "Why store the message?"

Sarah replied:

> "Because users may not always be online."

Message Queue

Chris explained:

A queue provides:

- * Reliability
- * Buffering
- * Scalability

If millions of messages arrive:

The queue absorbs the traffic.

Message Storage

The system stored:

- * Message ID
- * Sender ID
- * Receiver ID
- * Timestamp
- * Content
- * Status

Message History

When a user opens a conversation:

The system retrieves previous messages.

Sarah explained:

> "Reading history is different from delivering new messages."

Message Delivery States

The team designed states:

Sent

↓

Delivered

↓

Read

Emma smiled.

> "Like the check marks in messaging apps."

Delivery Guarantees

James asked:

> "Can we guarantee every message arrives exactly once?"

The room became quiet.

Chris smiled.

> "Distributed systems make this difficult."

At-Most-Once Delivery

The system sends once.

Problem:

Messages may disappear.

At-Least-Once Delivery

The system retries.

Problem:

Duplicate messages may appear.

Exactly-Once Delivery

The ideal solution.

One message.

One delivery.

No duplicates.

Chris explained:

> "Usually achieved using idempotency techniques and careful design."

Message Ordering

Another challenge appeared.

User sends:

Message 1

Message 2

Message 3

Network delays occur.

The receiver sees:

Message 2

Message 1

Message 3

Emma asked:

> "How do we fix ordering?"

Sequence Numbers

Sarah added:

Each message receives a sequence number.

Example:

Message 1

Sequence: 100

Message 2

Sequence: 101

The system reconstructs the correct order.

Group Chat

The team expanded the design.

One user sends:

"Hello everyone."

The message goes to:

100 people.

10,000 people.

1 million people.

Chris explained:

> "Large groups create fan-out challenges."

Fan-Out Strategies

Two approaches:

Fan-Out on Write

Copy messages to recipients immediately.

Advantages:

Fast reading.

Disadvantages:

Expensive writes.

Fan-Out on Read

Store once.

Fetch when users open chat.

Advantages:

Efficient storage.

Disadvantages:

More complex reads.

Media Messages

Users wanted:

- * Images
- * Videos
- * Documents

Sarah separated media handling.

Message:

↓

Media Service

↓

Object Storage

↓

CDN

Push Notifications

When users were offline:

The system sent notifications.

Example:

"New message received."

Architecture:

Message Event

↓

Notification Service

↓

Mobile Push Provider

Encryption

The security team reviewed the system.

Messages required:

- * Encryption in transit
- * Encryption at rest
- * Access control

End-to-End Encryption

Sarah explained:

> "With end-to-end encryption, only the sender and receiver can read the message."

Emma nodded.

> "The server stores encrypted data."

Scaling Globally

The company expanded worldwide.

The chat system needed:

- * Regional servers

- * Data replication
- * Low latency routing

Disaster Recovery

The team prepared:

- * Backups
- * Replication
- * Failover systems
- * Recovery procedures

Final Chat Architecture

The complete system:

Users

↓

WebSocket Gateway

↓

Connection Service

↓

Chat Service

↓

Message Queue

↓

Message Storage

↓

Notification Service

↓

Media Service

↓

CDN

The platform could now handle billions of conversations.

Wrapping Up

Maya summarized the real-time chat design.

- ✓ WebSockets
- ✓ Long Polling
- ✓ Connection Servers
- ✓ Presence System
- ✓ Heartbeats
- ✓ Message Queue
- ✓ Message Storage
- ✓ Message History
- ✓ Delivery States
- ✓ At-Most-Once Delivery
- ✓ At-Least-Once Delivery
- ✓ Exactly-Once Delivery
- ✓ Message Ordering
- ✓ Sequence Numbers
- ✓ Group Chat
- ✓ Fan-Out on Write
- ✓ Fan-Out on Read
- ✓ Media Service
- ✓ Push Notifications
- ✓ Encryption
- ✓ End-to-End Encryption
- ✓ Global Scaling
- ✓ Disaster Recovery

The team looked at the final architecture.

Emma smiled.

> "A chat application looks like a simple conversation window."

Chris nodded.

> "But behind every message is a distributed system solving impossible problems."

Sarah closed the design document.

> "The best systems disappear behind the experience."

The users would never see:

- * Connection management
- * Queues
- * Storage layers
- * Delivery guarantees
- * Replication systems

They would only see one thing:

A message arriving instantly.

And that was the true success of system design.

Part 7: System Design & Software Architecture

Chapter 6: Designing a Global Search Engine

The real-time chat system became a success.

Users could communicate instantly.

Messages arrived in milliseconds.

The company had mastered:

- * Real-time systems
- * Distributed messaging
- * Global scaling

But the next challenge was completely different.

Finding information.

The CEO entered the engineering meeting.

> "We want to build a search engine that can understand and organize the world's information."

The room became silent.

Emma smiled.

> "Search sounds simple."

Chris nodded.

> "A user types a word."

He wrote:

```

Search → Results

```

Then he added:

> "But behind that box is one of the most complex systems in technology."

Understanding the Requirements

Sarah started the design session.

> "What should users be able to do?"

The product team answered:

Users should be able to:

- * Search websites
- * Find documents
- * Search images
- * Get relevant results
- * Use autocomplete
- * Filter results
- * Receive fast responses

Sarah nodded.

> "These are functional requirements."

Non-Functional Requirements

Chris added:

The system must provide:

- * Extremely low latency
- * Massive scalability
- * High relevance
- * Global availability
- * Fault tolerance

Emma smiled.

> "The challenge is not finding information."

She paused.

> "The challenge is finding the right information."

Search Engine Architecture

The team created the first design:

Web Pages

↓

Crawler

↓

Processing Pipeline

↓

Index System

↓

Query Service

↓

Ranking Engine

↓

User Results

Web Crawlers

James asked:

> "How do we collect information from the internet?"

Sarah explained:

> "We use web crawlers."

Web Crawler

A crawler automatically visits websites.

It:

- * Downloads pages
- * Follows links
- * Collects information
- * Sends data for processing

Emma asked:

> "Does it visit every website?"

Chris smiled.

> "The internet is too large."

Crawl Strategy

The crawler decides:

- * Which pages to visit
- * How often to revisit
- * How much data to collect

Important factors:

- * Popularity
- * Freshness
- * Importance

URL Frontier

The crawler needs a queue of URLs.

Sarah introduced:

****URL Frontier****

It stores:

- * Pages waiting to be crawled
- * Crawl priorities
- * Scheduling information

Robots Rules

Some websites restrict crawling.

Chris explained:

> "A responsible crawler respects website rules."

The system checks:

- * Permissions
- * Crawl limits

Document Processing

After downloading pages:

The system extracts:

- * Text

- * Images
- * Links
- * Metadata

Emma asked:

> "How does the system understand the page?"

Sarah replied:

> "Through processing and analysis."

Inverted Index

The most important search structure appeared.

Chris wrote:

****Inverted Index****

Emma asked:

> "What is it?"

Chris explained:

Instead of:

Document → Words

We store:

Word → Documents

Example:

"cloud"

↓

Document 1

Document 5

Document 20

Emma smiled.

> "Searching becomes much faster."

Index Building

The system created indexes using:

- * Tokens
- * Words

- * Metadata
- * Links

The index became the foundation of search.

Distributed Index Storage

The index became enormous.

Billions of documents.

Trillions of words.

Sarah explained:

> "One machine cannot store everything."

Index Sharding

The index was divided.

Shard 1:

A-F

Shard 2:

G-M

Shard 3:

N-Z

Emma nodded.

> "Distributed storage again."

Query Processing

A user searches:

"best cloud architecture"

The request travels:

User

↓

Query Service

↓

Search Index

↓

Ranking System

↓

Results

Query Parsing

The system first understands the query.

It handles:

- * Spelling mistakes
- * Synonyms
- * Language differences
- * User intent

Ranking Algorithms

Finding documents is not enough.

Millions of results may exist.

The system must rank them.

Sarah explained:

> "Ranking decides what appears first."

Relevance Scoring

The system evaluates:

- * Keyword matching
- * Content quality
- * Freshness
- * Popularity
- * User behavior

Page Importance

Chris introduced:

****Link Analysis****

A page linked by many trusted websites may be more important.

Emma nodded.

```
> "The internet itself provides signals."

---

# Machine Learning Ranking

The AI team joined.

They built ranking models.

During training:

The model learns from:

* Search behavior
* User clicks
* Quality signals

During inference:

The model predicts:

* Best results
* Ranking order

---

# Autocomplete

Users expected suggestions while typing.

Example:

User types:

"cloud"

Suggestions:

* cloud computing
* cloud architecture
* cloud security

---

# Trie Data Structure

James asked:

> "How do we generate suggestions quickly?"

Sarah introduced:

**Trie**

A tree structure optimized for prefix searches.

---

# Search Suggestions
```

The system used:

- * Popular searches
- * User history
- * Trending topics

Caching Search Results

Popular searches happened millions of times.

Chris added:

****Query Cache****

Example:

"weather today"

↓

Cached Results

Emma smiled.

> "The system avoids repeated computation."

Personalization

Different users may receive different results.

Signals:

- * Location
- * Language
- * Previous searches
- * Preferences

Distributed Query Execution

Large searches required many machines.

The query was divided:

Machine 1:

Documents A-F

Machine 2:

Documents G-M

Machine 3:

Documents N-Z

Results combined:

↓

Final Ranking

Fault Tolerance

Search systems must survive failures.

The team added:

- * Replication
- * Backup indexes
- * Automatic recovery

Analytics System

The company tracked:

- * Search frequency
- * Popular queries
- * Failed searches
- * User satisfaction

These insights improved the system.

Security and Abuse Prevention

The system protected against:

- * Spam pages
- * Manipulated rankings
- * Malicious content

Final Search Architecture

The completed system:

Users

↓

Query Service

↓

Ranking Engine

↓

Search Index

↓

Distributed Storage

↓

Crawler System

↓

Document Processing

↓

Machine Learning Models

↓

Analytics Platform

Wrapping Up

Maya summarized the search engine design.

- ✅ Web Crawler
- ✅ URL Frontier
- ✅ Crawl Strategy
- ✅ Document Processing
- ✅ Inverted Index
- ✅ Index Building
- ✅ Index Sharding
- ✅ Query Processing
- ✅ Query Parsing
- ✅ Ranking Algorithms
- ✅ Relevance Scoring
- ✅ Link Analysis
- ✅ ML Ranking
- ✅ Autocomplete

- ✓ Trie
- ✓ Query Cache
- ✓ Personalization
- ✓ Distributed Query Execution
- ✓ Fault Tolerance
- ✓ Search Analytics
- ✓ Spam Prevention

The team looked at the final architecture.

Emma smiled.

> "A search box looks simple."

Chris nodded.

> "But behind every search is crawling, indexing, ranking, and distributed computation."

Sarah closed the design document.

> "Good system design hides complexity from users."

The user sees:

A search box.

The engineer sees:

A global information retrieval system.

And that difference is what separates simple applications from world-scale platforms.

Part 7: System Design & Software Architecture

Chapter 7: Designing a Global Ride-Sharing Platform

The search engine became one of the company's most advanced systems.

It could:

- * Crawl billions of pages
- * Index massive amounts of data
- * Rank information instantly

But the company wanted to solve a different problem.

A problem that happened in the physical world.

Transportation.

The CEO entered the engineering meeting.

> "We want to build a global ride-sharing platform."

Emma looked interested.

> "A system connecting passengers and drivers in real time."

Chris nodded.

> "And unlike search, the world is constantly moving."

Understanding the Requirements

Sarah started the discussion.

> "What should users be able to do?"

The product team answered:

Passengers:

- * Create accounts
- * Request rides
- * Track drivers
- * Pay digitally
- * Rate trips

Drivers:

- * Go online
- * Receive ride requests
- * Navigate routes
- * Complete trips

Sarah nodded.

> "These are functional requirements."

Non-Functional Requirements

Chris added:

The platform needs:

- * Real-time location updates

- * Low latency
- * High availability
- * Accurate matching
- * Global scalability
- * Safety and security

Emma smiled.

> "A delay of a few seconds can change the entire experience."

High-Level Architecture

The team designed:

Users

↓

Mobile Applications

↓

API Gateway

↓

Ride Services

↓

Location System

↓

Matching Engine

↓

Payment System

User Service

The platform needed user management.

It handled:

- * Passenger profiles
- * Driver profiles
- * Authentication
- * Ratings

Driver Availability

The system needed to know:

Who is online?

Who is available?

Who is currently driving?

Sarah created:

Driver State Service

States:

Offline

↓

Available

↓

Matched

↓

Driving

↓

Completed

Real-Time Location Tracking

The biggest challenge appeared.

Drivers move constantly.

Every few seconds:

New GPS location.

James asked:

> "Where do we store millions of moving vehicles?"

Chris smiled.

> "That is the location system problem."

Location Updates

Driver app sends:

Latitude

Longitude

Timestamp

↓

Location Service

Emma asked:

> "Every driver sends updates continuously?"

Sarah nodded.

> "Yes."

Geospatial Data

Traditional databases store:

Rows and columns.

But location requires:

- * Distance
- * Nearby objects
- * Geographic areas

Geospatial Indexing

Chris introduced:

****Geospatial Index****

It organizes locations efficiently.

The system can quickly answer:

"Find drivers near this passenger."

Geohashing

Sarah explained:

The map is divided into areas.

Each area receives a code.

Example:

Area A:

Drivers 1, 2, 3

Area B:

Drivers 4, 5, 6

Emma nodded.

> "Search nearby locations faster."

Quadtrees

Another approach appeared.

The map divides recursively.

Large area:

↓

Smaller areas

↓

Smaller areas

Chris explained:

> "Useful for spatial searching."

Ride Request Flow

A passenger requests a ride.

Flow:

Passenger

↓

Ride Service

↓

Matching Engine

↓

Nearby Drivers

↓

Driver Accepts

↓

Trip Begins

Matching Engine

The most important component.

It decides:

Which driver receives which request?

Factors:

- * Distance
- * Driver availability
- * Vehicle type
- * Estimated arrival time

Nearest Driver Problem

The simple approach:

Find closest driver.

But real life is more complicated.

The closest driver may:

- * Be busy soon
- * Have poor route access
- * Create traffic problems

Matching Algorithm

The system calculates:

- * Distance
- * ETA
- * Driver score
- * Traffic conditions

Then selects the best match.

Event-Driven Architecture

The team introduced events.

Examples:

Driver Online

↓

Event

Ride Requested

↓

Event

Trip Completed

↓

Event

Emma smiled.

> "Events keep services independent."

Message Queue

Millions of events flowed through the system.

The team added:

Message Queue

Benefits:

- * Reliability
- * Buffering
- * Scalability

Trip Service

The trip service managed:

- * Ride lifecycle
- * Pickup location
- * Destination
- * Status changes

Route Calculation

The platform needed navigation.

The routing system calculated:

- * Shortest path
- * Estimated arrival time
- * Traffic conditions

ETA Prediction

Machine learning improved predictions.

The model considered:

- * Historical traffic
- * Weather
- * Time of day
- * Road conditions

Surge Pricing

Demand changed constantly.

Example:

Many passengers.

Few drivers.

The price increased.

Sarah explained:

> "Dynamic pricing balances supply and demand."

Pricing Engine

It considered:

- * Distance
- * Time
- * Demand
- * Location
- * Vehicle type

Payment Integration

After the trip:

Trip Completed

↓

Payment Service

↓

Transaction System

↓

Receipt

The payment architecture reused previous lessons.

Ratings System

After each ride:

Passenger rates driver.

Driver rates passenger.

The system stored:

- * Ratings
- * Reviews
- * Feedback

Safety Features

The platform included:

- * Identity verification
- * Emergency features
- * Trip tracking
- * Fraud detection

Scaling Globally

The company expanded worldwide.

Different cities had:

- * Different traffic
- * Different regulations
- * Different demand patterns

Sarah designed:

Regional architectures.

Data Partitioning

The system separated data by:

- * Region
- * City
- * Geography

Emma nodded.

> "Local processing improves performance."

Real-Time Dashboard

Operations teams monitored:

- * Active drivers
- * Active passengers
- * Demand areas
- * System health

Final Ride-Sharing Architecture

The complete system:

Users

↓

Mobile Apps

↓

API Gateway

↓

Ride Service

↓

Matching Engine

↓

Location Service

↓

Geospatial Database

↓

Routing Service

↓

Payment System

↓

Analytics Platform

Wrapping Up

Maya summarized the ride-sharing system.

✅ User Service

- ✓ Driver State Management
- ✓ Real-Time Location Tracking
- ✓ GPS Updates
- ✓ Geospatial Indexing
- ✓ Geohashing
- ✓ Quadtrees
- ✓ Ride Request Flow
- ✓ Matching Engine
- ✓ Matching Algorithms
- ✓ Event-Driven Architecture
- ✓ Message Queue
- ✓ Trip Service
- ✓ Route Calculation
- ✓ ETA Prediction
- ✓ Machine Learning
- ✓ Surge Pricing
- ✓ Pricing Engine
- ✓ Payment Integration
- ✓ Ratings System
- ✓ Safety Features
- ✓ Regional Scaling

The team reviewed the architecture.

Emma smiled.

> "A ride-sharing app looks like a simple map."

Chris nodded.

> "But behind that map is a global distributed system tracking millions of moving objects."

Sarah closed the design document.

> "The hardest systems are often the ones that interact with the real world."

The next challenge was waiting.

A system where billions of pieces of data move continuously.

A system where timing, reliability, and scale matter more than ever.

A global Internet of Things platform.

Part 7: System Design & Software Architecture

Chapter 8: Designing a Global IoT Platform

The ride-sharing platform became a global success.

Millions of drivers and passengers used it every day.

The company had solved:

- * Real-time location tracking
- * Distributed matching
- * Event processing
- * Global scaling

But a new opportunity appeared.

Connected devices.

The CEO entered the engineering meeting.

> "The world is becoming connected."

He displayed a presentation:

- * Smart homes
- * Industrial machines
- * Healthcare devices
- * Smart cities
- * Connected vehicles

> "We want to build a global IoT platform."

Emma looked at the screen.

> "Billions of devices sending data."

Chris nodded.

> "And unlike users, devices never stop talking."

Understanding the Requirements

Sarah started the design session.

> "What should the platform support?"

The product team answered:

Devices should be able to:

- * Connect securely
- * Send data
- * Receive commands
- * Report status
- * Update software
- * Trigger actions

Sarah nodded.

> "These are functional requirements."

Non-Functional Requirements

Chris added:

The platform must provide:

- * Massive scalability
- * Low latency
- * High reliability
- * Device security
- * Real-time processing
- * Long-term data storage

Emma smiled.

> "A billion devices cannot be managed like a billion users."

High-Level IoT Architecture

The team created:

Devices

↓

Communication Layer

↓

IoT Platform

↓

Processing Systems

↓

Storage

↓

Applications

Device Registration

The first challenge:

How does a device join the platform?

Sarah created:

****Device Registry****

It stores:

- * Device ID
- * Device type
- * Owner
- * Configuration
- * Security credentials

Device Identity

James asked:

> "How do we know a device is real?"

Chris explained:

Each device receives:

- * Unique identity
- * Certificate
- * Authentication credentials

Emma nodded.

> "Every device must prove who it is."

Device Authentication

The security team designed:

Device

↓

Authentication Service

↓

Access Granted

Methods:

- * Certificates
- * Tokens
- * Secure keys

Communication Protocols

The team discussed communication.

Different devices have different needs.

Examples:

- * HTTP
- * WebSockets
- * MQTT
- * CoAP

MQTT

Sarah introduced the most common IoT protocol.

****MQTT****

Emma asked:

> "Why use MQTT?"

Chris explained:

> "It is lightweight and designed for unreliable networks."

Publish and Subscribe Model

MQTT uses:

Publisher

↓

Broker

↓

Subscriber

Example:

Temperature Sensor

↓

MQTT Broker

↓

Monitoring System

IoT Message Broker

The broker manages communication.

Responsibilities:

- * Receive messages
- * Route messages
- * Manage subscriptions
- * Handle connections

Device Telemetry

Devices continuously send information.

Examples:

Temperature:

25°C

Vehicle:

Speed 70 km/h

Factory machine:

Running

Sarah explained:

> "This data is called telemetry."

Telemetry Pipeline

The data flow:

Device

↓

Message Broker

↓

Stream Processing

↓

Storage

↓

Analytics

Stream Processing

Billions of events arrived every second.

The system needed immediate processing.

Chris introduced:

****Stream Processing****

Real-Time Analytics

The system detected:

- * Abnormal temperatures
- * Machine failures
- * Security threats

Emma smiled.

> "The system reacts while data is arriving."

Time-Series Database

IoT data has a special pattern.

Example:

Temperature over time.

Sarah introduced:

****Time-Series Database****

Designed for:

- * Timestamped data
- * High write volume
- * Historical analysis

Data Storage Strategy

Not all data was stored the same way.

Recent data:

Fast storage.

Old data:

Long-term storage.

Emma nodded.

> "Storage depends on usage."

Device Commands

The platform also sent instructions.

Example:

Turn on machine.

Update configuration.

Restart device.

Flow:

Application

↓

Command Service

↓

Device

Command Delivery

The system needed to know:

Was the command:

* Sent?

* Received?

* Completed?

Sarah created command states.

Created

↓

Delivered

↓

Executed

Edge Computing

Some devices needed instant decisions.

Example:

Factory safety system.

Waiting for cloud response was too slow.

Chris introduced:

****Edge Computing****

Edge Devices

Processing happens near the device.

Benefits:

- * Lower latency
- * Reduced network usage
- * Faster decisions

Cloud and Edge Together

The architecture became:

Device

↓

Edge Processing

↓

Cloud Platform

↓

Analytics

Emma smiled.

> "Not everything needs to travel to the cloud."

Digital Twins

The team created virtual representations.

Example:

Real machine:

Factory Robot #25

↓

Digital Twin

Virtual model containing:

- * Current state
- * History
- * Configuration

Device Monitoring

Operations teams needed visibility.

Dashboards showed:

- * Connected devices
- * Errors
- * Performance
- * Location

Firmware Updates

Millions of devices needed software updates.

Sarah designed:

Over-The-Air Updates (OTA)

OTA Update Process

Cloud

↓

Update Package

↓

Device

↓

Installation

↓

Verification

Emma nodded.

> "Updating devices without touching them physically."

IoT Security

The security team reviewed risks.

Threats:

- * Unauthorized devices
- * Data theft
- * Device takeover

Protection:

- * Encryption
- * Authentication
- * Secure updates
- * Access control

Device Lifecycle Management

Devices have lifecycles.

Stages:

Manufactured

↓

Registered

↓

Active

↓

Updated

↓

Retired

Scaling Challenges

The platform needed to support:

Millions of devices.

Then:

Billions of devices.

Solutions:

- * Horizontal scaling
- * Partitioning
- * Regional deployment
- * Distributed messaging

Final IoT Architecture

The completed platform:

Devices

↓

MQTT Broker

↓

IoT Gateway

↓

Stream Processing

↓

Time-Series Database

↓

Analytics Platform

↓

Applications

↓

Digital Twin System

Wrapping Up

Maya summarized the IoT platform design.

- ✅ Device Registry
- ✅ Device Identity
- ✅ Device Authentication

- ✓ MQTT
- ✓ Publish/Subscribe Model
- ✓ Message Broker
- ✓ Telemetry
- ✓ Telemetry Pipeline
- ✓ Stream Processing
- ✓ Real-Time Analytics
- ✓ Time-Series Database
- ✓ Device Commands
- ✓ Command Delivery
- ✓ Edge Computing
- ✓ Digital Twins
- ✓ Device Monitoring
- ✓ OTA Updates
- ✓ IoT Security
- ✓ Device Lifecycle Management
- ✓ Global Scaling

The team reviewed the final architecture.

Emma smiled.

> "A smart device looks simple."

Chris nodded.

> "But behind every sensor is a complete distributed system."

Sarah closed the design document.

> "The future is not only software running on computers."

She looked at the connected devices around the room.

> "The future is software connecting the physical world."

The team had designed:

- * Social platforms
- * Payment systems
- * Streaming services
- * Chat systems
- * Search engines
- * Ride-sharing platforms
- * IoT systems

But one final challenge remained.

A system where intelligence itself becomes part of the architecture.

Artificial Intelligence at massive scale.

Part 7: System Design & Software Architecture

Chapter 9: Designing a Global AI Platform

The IoT platform changed the way the company interacted with the physical world.

Billions of devices were connected.

Massive amounts of data flowed every second.

But a new question appeared:

> "How can we make systems intelligent?"

The CEO entered the engineering meeting.

> "We want to build an AI platform that can power intelligent products across the company."

Emma looked at the architecture board.

> "So we are not building one AI feature."

Chris nodded.

> "We are building the infrastructure that allows thousands of AI applications to exist."

Understanding the Requirements

Sarah started the design discussion.

> "What should the AI platform provide?"

The product teams answered:

They need to:

- * Train machine learning models
- * Store training data
- * Deploy models
- * Make predictions
- * Monitor AI performance
- * Improve models continuously

Sarah nodded.

> "These are functional requirements."

Non-Functional Requirements

Chris added:

The platform needs:

- * High scalability
- * Fast inference
- * Reliable model serving
- * Data security
- * Reproducibility
- * Cost efficiency

Emma smiled.

> "AI systems are not only about models."

She pointed at the architecture.

> "They are about everything around the models."

AI Platform Architecture

The team created:

Data Sources

↓

Data Pipeline

↓

Feature Engineering

↓

Feature Store

↓

Model Training

↓

Model Registry

↓

Model Deployment

↓

Inference Service

↓

Monitoring

Data Collection

The AI system needed information.

Sources included:

- * User behavior
- * Application logs
- * Sensors
- * Transactions
- * Images
- * Text

Sarah explained:

> "Models learn from data."

Data Pipeline

Raw data was messy.

The system needed:

- * Collection
- * Cleaning
- * Transformation
- * Validation

Chris explained:

> "A model is only as good as the data it receives."

Data Processing

The team created processing jobs.

Tasks:

- * Remove duplicates
- * Fix missing values

- * Normalize information
- * Prepare datasets

Emma nodded.

> "Preparing data is a major part of AI engineering."

Feature Engineering

Sarah introduced:

****Features****

Emma asked:

> "What are features?"

Chris explained:

> "Features are meaningful inputs that help a model make decisions."

Examples:

For recommendation:

- * Watch history
- * User interests
- * Time spent

For fraud detection:

- * Transaction amount
- * Location
- * Device behavior

Feature Store

The team needed a central place for features.

Sarah created:

****Feature Store****

It provided:

- * Feature storage
- * Feature reuse
- * Consistency between training and production

Training Environment

The machine learning team needed powerful computers.

Chris introduced:

Training Infrastructure

It included:

- * GPUs
- * Distributed computing
- * Experiment tracking

Model Training

The AI team trained a model.

During training:

The model learns patterns from data.

Example:

Input:

Millions of examples

↓

Learning process

↓

Trained model

Machine Learning Models

Different problems required different models.

Examples:

- * Recommendation models
- * Fraud detection models
- * Language models
- * Image recognition models

Overfitting Problem

Emma asked:

> "Can a model learn too much?"

Sarah nodded.

> "Yes."

She explained:

> "Overfitting happens when a model memorizes training data instead of learning general patterns."

Model Evaluation

The team measured performance.

Metrics included:

- * Accuracy
- * Precision
- * Recall
- * F1 Score

Chris explained:

> "Different problems require different measurements."

Bias Detection

The AI team reviewed predictions.

They discovered:

Some groups received less accurate results.

Sarah explained:

> "Models can learn bias from data."

The team added:

- * Bias testing
- * Better datasets
- * Fairness evaluation

Model Registry

The team created a system to store models.

It tracked:

- * Model versions
- * Training data
- * Performance
- * Deployment status

Emma smiled.

> "Like source control for models."

Model Deployment

The company needed models in production.

The flow:

Model

↓

Deployment Pipeline

↓

Inference Service

↓

Applications

Inference

Chris explained:

Training:

Model learns.

Inference:

Model makes predictions.

Example:

User watches a video.

↓

Recommendation model

↓

Suggested videos

Model Serving

The team built:

****Model Serving Infrastructure****

It handled:

- * Prediction requests
- * Scaling
- * Version management
- * Response times

Real-Time Inference

Some predictions needed instant responses.

Examples:

- * Fraud detection
- * Search ranking
- * Recommendations

The system required:

- * Low latency
- * Fast computation
- * Reliable APIs

Batch Inference

Other predictions did not need immediate results.

Examples:

- * Weekly reports
- * Large analysis jobs

The system processed them in batches.

Vector Databases

The AI team introduced a new requirement.

Modern AI systems needed similarity search.

Sarah explained:

> "Traditional databases are not always ideal for finding similar concepts."

Embeddings

The model converted information into vectors.

Example:

Text

↓

Vector representation

↓

Similarity search

Emma nodded.

> "Meaning becomes searchable."

Vector Database

The system stored:

- * Embeddings
- * Metadata
- * Relationships

Used for:

- * Semantic search
- * Recommendations
- * AI assistants

MLOps

The engineering team combined:

Machine Learning

*

Operations

Chris introduced:

****MLOps****

MLOps Pipeline

The automated workflow:

Data

↓

Training

↓

Testing

↓

Deployment

↓

Monitoring

↓

Improvement

Model Monitoring

The team watched:

- * Accuracy changes
- * Latency
- * Data changes
- * Prediction quality

Model Drift

After months, user behavior changed.

The model became less accurate.

Sarah explained:

> "This is called model drift."

The solution:

- * Retrain models
- * Update datasets
- * Monitor changes

AI Security

The security team reviewed risks.

Threats:

- * Data leakage
- * Model attacks
- * Unauthorized access

Protection:

- * Encryption
- * Access control
- * Secure pipelines

Scaling AI Systems

The platform needed to support:

Thousands of models.

Millions of predictions.

Solutions:

- * Distributed computing
- * GPU clusters
- * Autoscaling
- * Caching

Final AI Platform Architecture

The completed system:

Data Sources

↓

Data Pipeline

↓

Feature Store

↓

Training Infrastructure

↓

Model Registry

↓

MLOps Pipeline

↓

Model Serving

↓

Inference APIs

↓

Applications

↓

Monitoring

Wrapping Up

Maya summarized the AI platform design.

- ✓ Data Pipeline
- ✓ Data Processing
- ✓ Feature Engineering
- ✓ Feature Store
- ✓ Model Training
- ✓ Training Infrastructure
- ✓ Model Evaluation
- ✓ Precision
- ✓ Recall
- ✓ Bias Detection
- ✓ Model Registry
- ✓ Model Deployment
- ✓ Inference
- ✓ Model Serving
- ✓ Real-Time Inference
- ✓ Batch Inference
- ✓ Embeddings
- ✓ Vector Database
- ✓ MLOps
- ✓ Model Monitoring
- ✓ Model Drift
- ✓ AI Security
- ✓ AI Scaling

The team reviewed the final architecture.

Emma smiled.

> "An AI application looks like a simple prediction."

Chris nodded.

> "But behind every prediction is a complete engineering system."

Sarah closed the design document.

> "The future belongs to engineers who understand both software systems and intelligence."

The company had now designed systems for:

- * People
- * Money
- * Media
- * Communication
- * Search
- * Transportation
- * Devices
- * Intelligence

But one final lesson remained.

How do all these systems work together as one massive digital ecosystem?

Part 7: System Design & Software Architecture

Chapter 10: The Complete Enterprise Architecture

The AI platform became the company's most advanced system.

It could:

- * Train intelligent models
- * Make predictions
- * Improve products
- * Automate decisions

The company had built many powerful platforms.

But something became obvious.

Each system was successful on its own.

The real challenge was connecting everything together.

The CEO called a meeting.

On the screen appeared a new question:

"How do we build one unified digital ecosystem?"

Emma looked at the architecture diagrams covering the entire wall.

Social Platform.

Payment System.

Streaming Platform.

Chat System.

Search Engine.

Ride-Sharing Platform.

IoT Platform.

AI Platform.

Chris smiled.

> "We have built many systems."

Sarah nodded.

> "Now we need an architecture that connects them."

Enterprise Architecture

Sarah wrote:

****Enterprise Architecture****

She explained:

> "Enterprise architecture defines how an organization's technology systems work together."

It includes:

- * Applications
- * Data
- * Infrastructure
- * Security
- * Business processes

The Big Picture

The complete ecosystem:

Users

↓

Applications

↓

API Platform

↓

Business Services

↓

Data Platforms

↓

AI Systems

↓

Cloud Infrastructure

↓

Security + Governance

Emma smiled.

> "Everything becomes connected."

Architecture Layers

Chris divided the architecture into layers.

Presentation Layer

What users interact with:

- * Mobile apps
- * Websites
- * Dashboards

Application Layer

Business logic:

- * User services
- * Payment services
- * Recommendation services

Data Layer

Information systems:

- * Databases
- * Data warehouses
- * Data lakes

Infrastructure Layer

The foundation:

- * Cloud resources
- * Networks
- * Compute systems

API Ecosystem

The company had hundreds of services.

Sarah asked:

> "How do systems communicate?"

Chris answered:

> "Through APIs."

API Gateway

The company created a central gateway.

It handled:

- * Authentication
- * Routing
- * Rate limiting
- * Monitoring

Emma nodded.

> "One controlled entrance to many systems."

API Management

The team created rules for APIs:

- * Versioning
- * Documentation
- * Security policies
- * Usage limits

Integration Architecture

Different systems needed to exchange information.

Examples:

Payment System

↓

Order System

↓

Notification System

↓

Analytics Platform

Event-Driven Enterprise

Sarah introduced:

****Event-Driven Architecture****

Instead of direct communication:

Service A

↓

Event

↓

Multiple Services

Enterprise Event Bus

The company created a central event platform.

Events included:

- * User Created
- * Payment Completed
- * Order Shipped
- * Device Connected
- * Model Updated

Why Events Matter

Chris explained:

> "Events reduce dependency between systems."

Emma nodded.

> "Services can evolve independently."

Data Architecture

The company generated enormous amounts of data.

Sarah designed:

A modern data platform.

Data Lake

Raw information was stored in one place.

Examples:

- * Logs
- * Videos
- * Device data
- * User activity

Data Warehouse

Processed business data was stored for analysis.

Used for:

- * Reports
- * Dashboards
- * Business decisions

Data Pipeline

The flow:

Sources

↓

Collection

↓

Processing

↓

Storage

↓

Analytics

↓

AI Models

Data Governance

Maya asked:

> "Who controls all this data?"

Sarah explained:

Data governance defines:

- * Ownership
- * Quality rules
- * Access policies
- * Compliance

Master Data Management

The company had duplicate information.

Examples:

Different versions of:

- * Customer profiles
- * Products
- * Locations

The team created:

****Master Data Management****

A trusted source of important data.

Security Architecture

The entire ecosystem required protection.

Chris created:

A security architecture.

Zero Trust Enterprise

The principle:

> "Never trust automatically. Always verify."

Every request required:

- * Identity verification
- * Authorization
- * Monitoring

Identity Platform

The company created centralized identity management.

It handled:

- * Employees
- * Customers
- * Services
- * Devices

Secrets and Encryption

All systems used:

- * Encryption
- * Key management
- * Secrets storage

Governance and Compliance

Large organizations require rules.

The company created:

- * Security standards
- * Architecture reviews
- * Audit processes

Architecture Patterns

Senior engineers used proven patterns.

Sarah reviewed them.

Layered Architecture

Separates responsibilities.

Example:

Presentation

↓

Business Logic

↓

Data Access

Microservices Pattern

Small independent services.

Benefits:

- * Independent deployment
- * Scalability
- * Team ownership

Event-Driven Pattern

Systems communicate through events.

Benefits:

- * Loose coupling
- * Flexibility
- * Scalability

CQRS Pattern

Chris introduced:

****Command Query Responsibility Segregation****

Emma asked:

> "What does it mean?"

Chris explained:

Separate:

Writing data

from

Reading data

Event Sourcing

Another pattern appeared.

Instead of storing only the current state:

The system stores every change.

Example:

Account Created

↓

Money Added

↓

Payment Sent

Strangler Pattern

The company still had old systems.

Sarah explained:

> "We replace old systems gradually."

New services slowly take over.

Architecture Decision Records (ADR)

Important decisions were documented.

Example:

Decision:

Use event-driven architecture.

Reason:

Improve scalability.

Trade-off:

Higher complexity.

Technical Trade-Offs

Emma asked:

> "Is there a perfect architecture?"

Everyone laughed.

Chris answered:

> "No."

Every choice has trade-offs.

Examples:

Performance vs Maintainability

Consistency vs Availability

Speed vs Cost

Simplicity vs Flexibility

The Role of a Software Architect

Maya asked Sarah:

> "What does an architect really do?"

Sarah answered:

> "An architect does not just draw diagrams."

They:

- * Understand business goals
- * Make technical decisions
- * Manage trade-offs
- * Guide engineering teams
- * Protect long-term quality

The Final Enterprise Architecture

The complete ecosystem:

Users

↓

Applications

↓

API Gateway

↓

Microservices

↓

Event Bus

↓

Data Platform

↓

AI Platform

↓

Cloud Infrastructure

↓

Security + Governance

Final System Design Journey

Maya summarized everything the team had learned.

Foundations

✓ Requirements

✓ Scalability

✓ Reliability

✓ Performance

Distributed Systems

✓ Caching

✓ Databases

✓ Queues

✓ Replication

✓ Sharding

Modern Platforms

✓ Cloud

✓ Containers

✓ Kubernetes

✓ DevOps

Advanced Systems

✓ Social Networks

✓ Payments

✓ Streaming

✓ Chat

✓ Search

✓ Ride Sharing

✓ IoT

✓ AI Platforms

Architecture

✓ Enterprise Architecture

✓ API Ecosystems

✓ Event-Driven Systems

✓ Data Platforms

✓ Security Architecture

✓ Design Patterns

The team stood in front of the final architecture diagram.

It covered the entire wall.

Emma smiled.

> "We started by learning how to build applications."

Chris nodded.

> "Then we learned how to build systems."

Sarah added:

> "Now we understand how organizations build technology ecosystems."

The CEO looked at the architecture.

> "What is the most important technology we created?"

Everyone expected an answer about cloud, AI, or software.

Sarah smiled.

> "Engineering judgment."

The Final Lesson

Technology changes.

Tools change.

Frameworks change.

Platforms change.

But great engineers understand:

- * Problems before solutions.
- * Trade-offs before decisions.
- * Users before systems.
- * Principles before tools.

A senior engineer is not someone who knows every technology.

A senior engineer knows:

****How to design systems that survive change.****

And that is the foundation of software architecture.